

 SOFTWARE ENGINEERING COURSE

LECTURE 5

Software Modelling

Micheal Tuape, D.Sc



MODULE 1 SCHEDULE

Week	Dates	Topic	SE Chapters
Week 1	September 2 — September 6, 2024	Course Introduction	None
Week 2	September 9 — September 13, 2024	Introduction to Software Engineering	Ch. 1
Week 3	September 16 — September 20, 2024	Software Development Life Cycle & Methodologies	Ch. 2
Week 4	September 23 — September 28, 2024	Software Development Processes	Ch. 3
Week 5	September 30 — October 4, 2024	Requirements Engineering	Ch. 4
Week 6	October 7 — October 11, 2024	Software Modelling, Coding and Dev.	Ch. 5
Week 7	October 14 — October 18, 2024	Software Architectural Design	Ch. 6





LEARNING OBJECTIVES

Context models

Interaction models

Structural models

Behavioral models

Model-driven engineering



SYSTEM MODELING

- »» System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.
- »» System modeling has now come to mean representing a system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML).
- »» System modelling helps the analyst to understand the functionality of the system and models are used to communicate with customers.





EXISTING AND PLANNED SYSTEM MODELS

- »» Models of the existing system are used during requirements engineering. They help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses. These then lead to requirements for the new system.
- »» Models of the new system are used during requirements engineering to help explain the proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation.
- »» In a model-driven engineering process, it is possible to generate a complete or partial system implementation from the system model.





SYSTEM PERSPECTIVES

- »» An external perspective, where you model the context or environment of the system.
- »» An interaction perspective, where you model the interactions between a system and its environment, or between the components of a system.
- »» A structural perspective, where you model the organization of a system or the structure of the data that is processed by the system.
- »» A behavioral perspective, where you model the dynamic behavior of the system and how it responds to events.





UML

- » Software Engineers relentlessly seek techniques to
- automate the production of software
 - to improve quality
 - reduce cost and time-to-market.

These techniques include

- component technology,
- visual programming,
- patterns and frameworks.
- techniques to manage the complexity of systems as they increase in scope and scale.





UML

- »» Software Engineers also strive to solve recurring architectural problems, such as
 - physical distribution,
 - concurrency,
 - replication,
 - security,
 - load balancing and fault tolerance.
- »» the development for the World Wide Web, while making some things simpler, has exacerbated these architectural problems. The Unified Modeling Language (UML) was designed to respond to these needs.





UML

- »» The primary goals in the design of UML is to provide stakeholders with:
- A ready-to-use, expressive visual modeling language so they can develop and exchange meaningful models.
 - Extensible and specialized mechanisms to extend the core concepts.
 - An independent avenue for particular programming languages and development processes.
 - A formal basis for understanding the modeling language.
 - The support for higher-level development concepts such as collaborations, frameworks, patterns and components.
 - The bases to integrate best practices.





UML

- »» UML offers lots of different diagrams (models) to get used to. This is because we look at a system from many different viewpoints. During software development we have many stakeholders with different roles.
- »» Stakeholders in SE:
 - Analysts
 - Designers
 - Coders
 - Testers
 - QA
 - The Customer
 - Technical Authors





UML DIAGRAM TYPES

»» Structure diagrams

Structure diagrams show the static structure of the system and its parts on different abstraction and implementation levels and how they are related to each other.

- Represent the meaningful concepts of a system,
- It may include abstract, real world and
- Implementation concepts





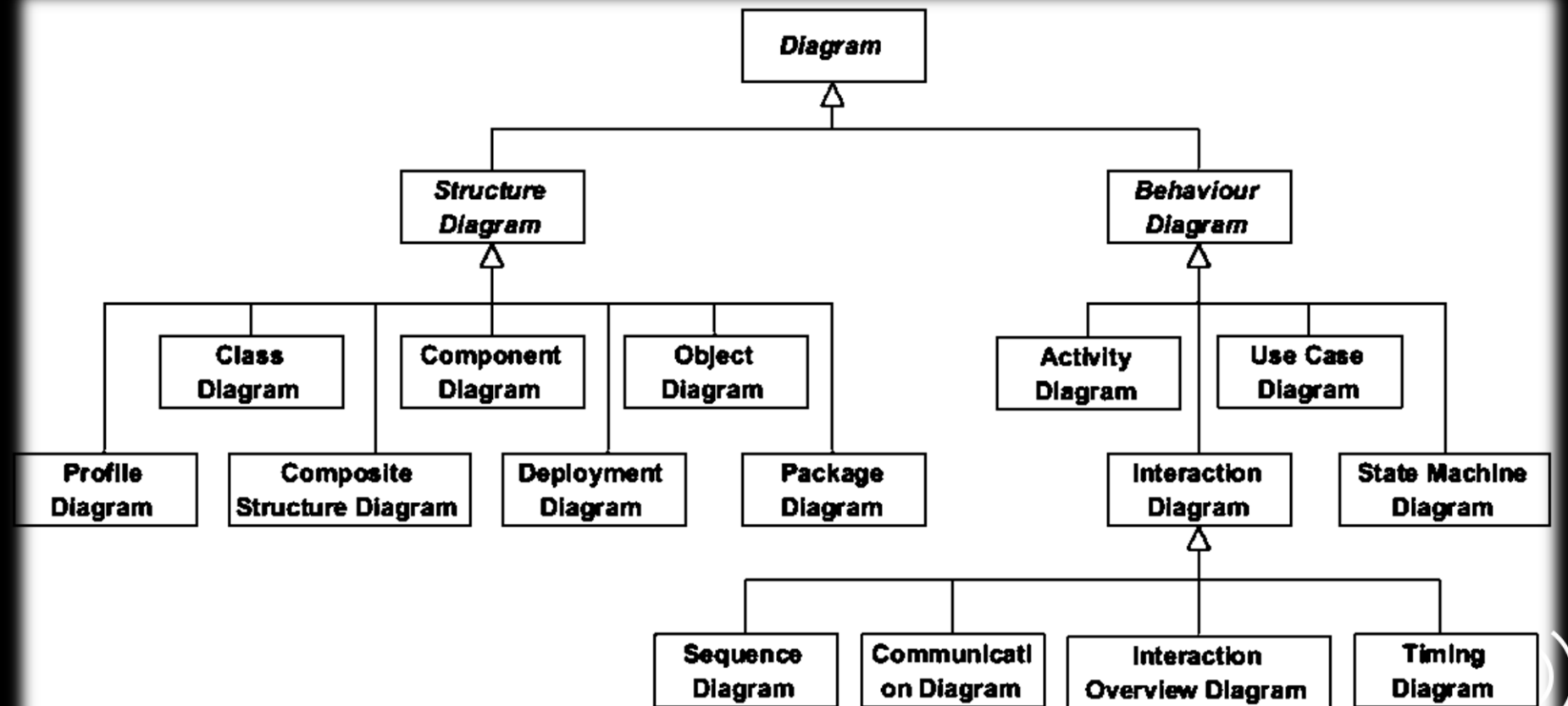
UML DIAGRAM TYPES

»» Behavior diagrams

Behavior diagrams show the dynamic behavior of the objects in a system, which can be described as a series of changes to the system over time,



UML DIAGRAM TYPES





UML DIAGRAM TYPES

- »» Activity diagrams, which show the activities involved in a process or in data processing.
- »» Use case diagrams, which show the interactions between a system and its environment.
- »» Sequence diagrams, which show interactions between actors and the system and between system components.
- »» Class diagrams, which show the object classes in the system and the associations between these classes.
- »» State diagrams, which show how the system reacts to internal and external events.





USE OF GRAPHICAL MODELS

- As a means of facilitating discussion about an existing or proposed system
 - Incomplete and incorrect models are OK as their role is to support discussion.
- As a way of documenting an existing system
 - Models should be an accurate representation of the system but need not be complete.
- As a detailed system description that can be used to generate a system implementation
 - Models have to be both correct and complete.





SYSTEM BOUNDARIES

- »» System boundaries are established to define what is inside and what is outside the system.
 - They show other systems that are used or depend on the system being developed.
- »» The position of the system boundary has a profound effect on the system requirements.
- »» Defining a system boundary is a political judgment
 - There may be pressures to develop system boundaries that increase / decrease the influence or workload of different parts of an organization.





PROCESS PERSPECTIVE

- »» Context models simply show the other systems in the environment, not how the system being developed is used in that environment.
- »» Process models reveal how the system being developed is used in broader business processes.
- »» UML activity diagrams may be used to define business process models.





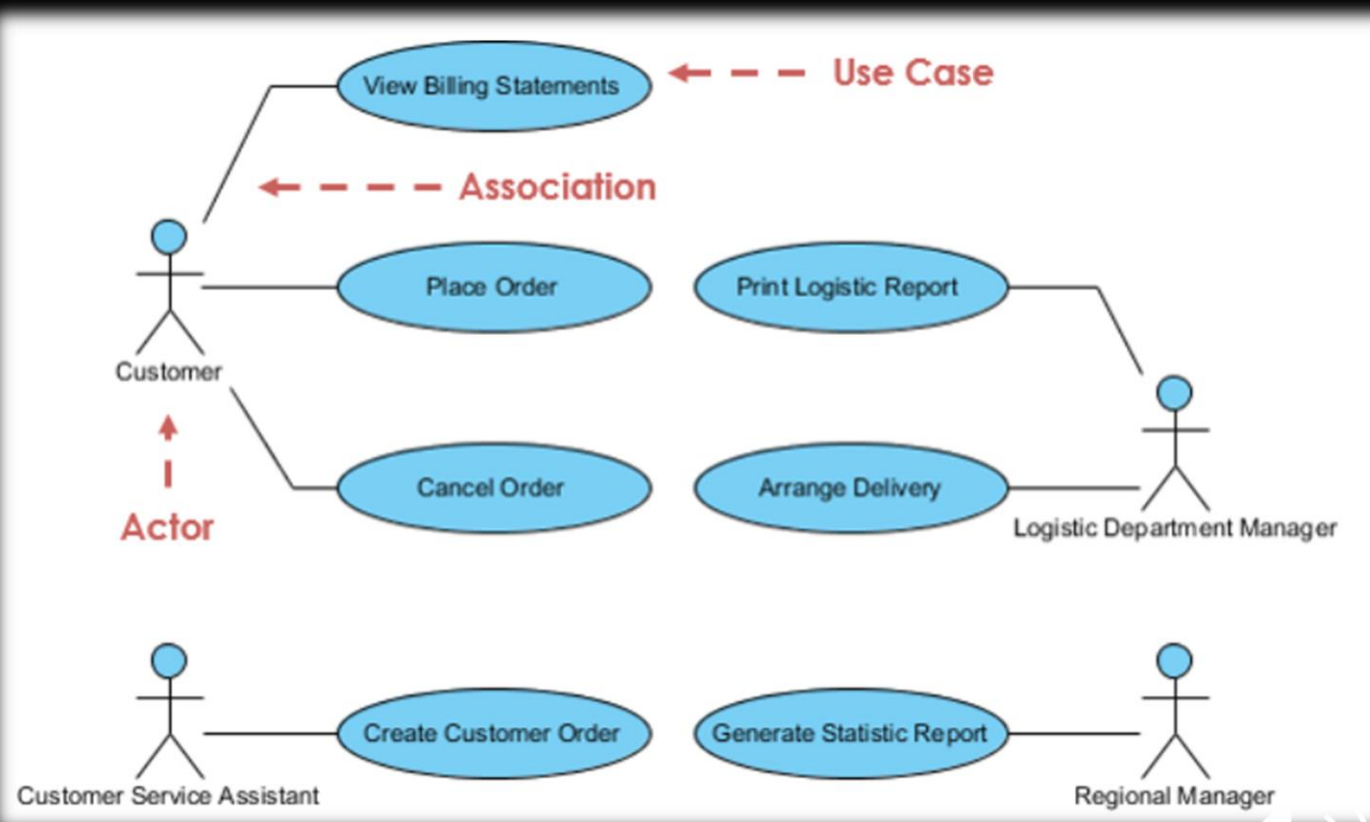
INTERACTION MODELS

- »» Modeling user interaction is important as it helps to identify user requirements.
- »» Modeling system-to-system interaction highlights the communication problems that may arise.
- »» Modeling component interaction helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.
- »» Use case diagrams and sequence diagrams may be used for interaction modeling.



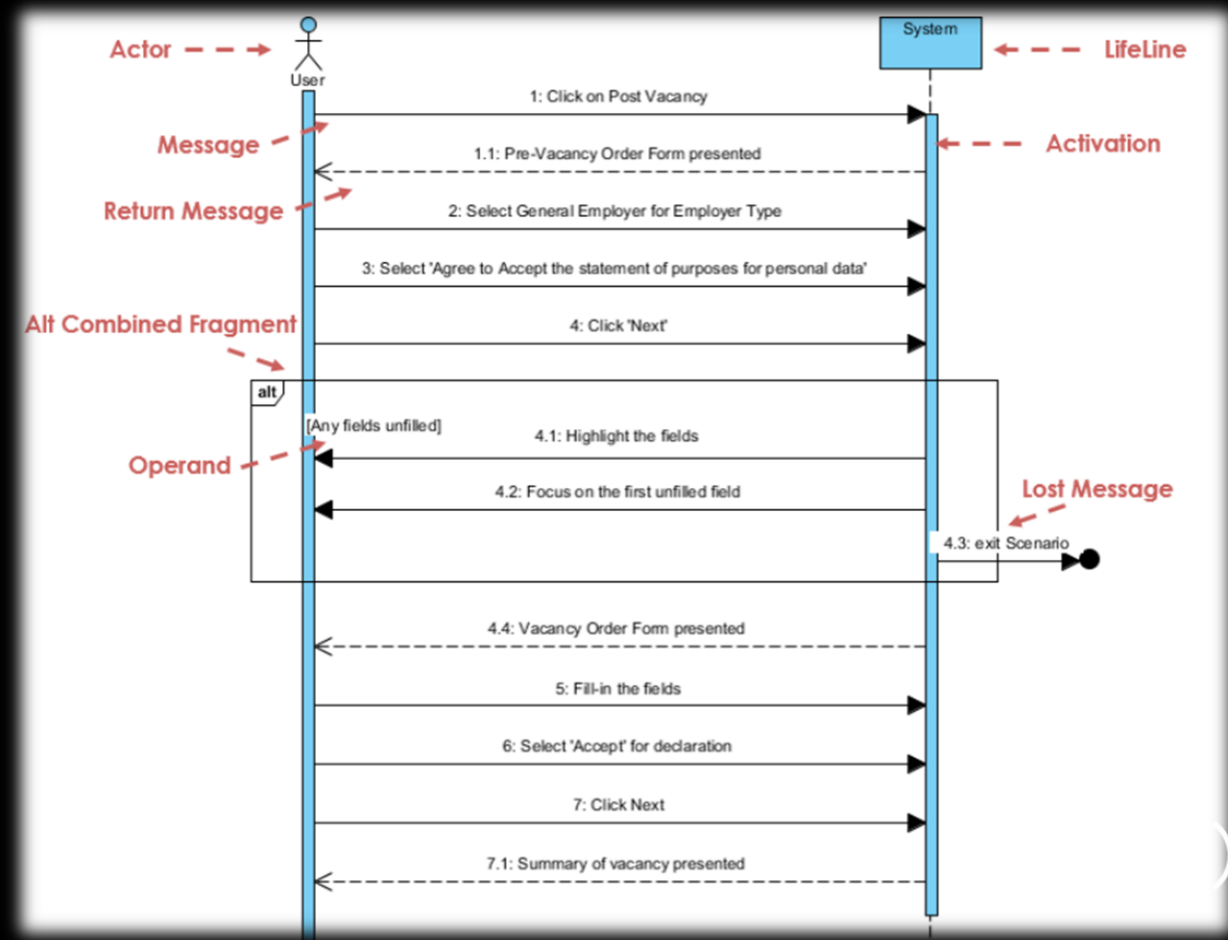
USE CASE MODELING

- » Use cases were developed originally to support requirements elicitation and now incorporated into the UML.
- » Each use case represents a discrete task that involves external interaction with a system.
- » Actors in a use case may be people or other systems.
- » Represented diagrammatically to provide an overview of the use case and in a more detailed textual form.



SEQUENCE DIAGRAMS

- Sequence diagrams are part of the UML and are used to model the interactions between the actors and the objects within a system.
- A sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance.
- The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these.
- Interactions between objects are indicated by annotated arrows.





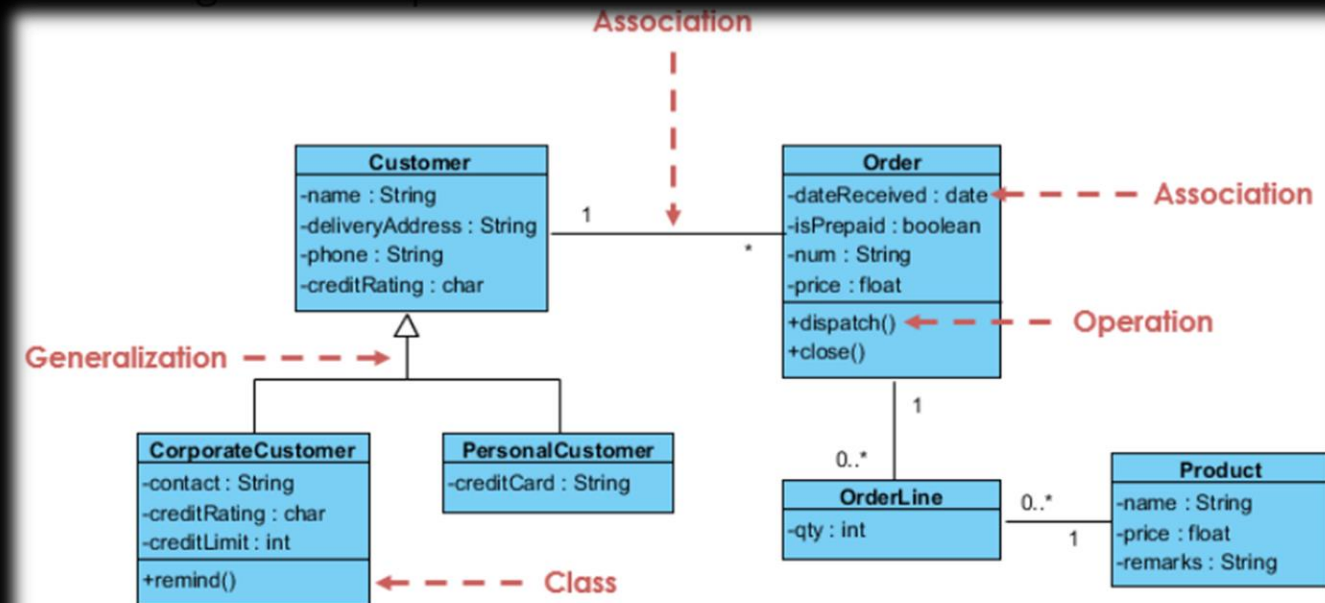
STRUCTURAL MODELS

- »» Structural models of software display the organization of a system in terms of the components that make up that system and their relationships.
- »» Structural models may be static models, which show the structure of the system design, or dynamic models, which show the organization of the system when it is executing.
- »» You create structural models of a system when you are discussing and designing the system architecture.



CLASS DIAGRAMS

- Class diagrams are used when developing an object-oriented system model to show the classes in a system and the associations between these classes.
- An object class can be thought of as a general definition of one kind of system object.
- An association is a link between classes that indicates that there is some relationship between these classes.
- When you are developing models during the early stages of the software engineering process, objects represent something in the real world, such as a patient, a prescription, doctor, etc.





GENERALIZATION

- »» Generalization is an everyday technique that we use to manage complexity.
- »» Rather than learn the detailed characteristics of every entity that we experience, we place these entities in more general classes and learn the characteristics of these classes.
- »» This allows us to infer that different members of these classes have some common characteristics.





BEHAVIORAL MODELS

- »» Behavioral models are models of the dynamic behavior of a system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment.
- »» You can think of these stimuli as being of two types:
 - Data Some data arrives that has to be processed by the system.
 - Events Some event happens that triggers system processing. Events may have associated data, although this is not always the case.





DATA-DRIVEN MODELING

- »» Many business systems are data-processing systems that are primarily driven by data. They are controlled by the data input to the system, with relatively little external event processing.
- »» Data-driven models show the sequence of actions involved in processing input data and generating an associated output.
- »» They are particularly useful during the analysis of requirements as they can be used to show end-to-end processing in a system.





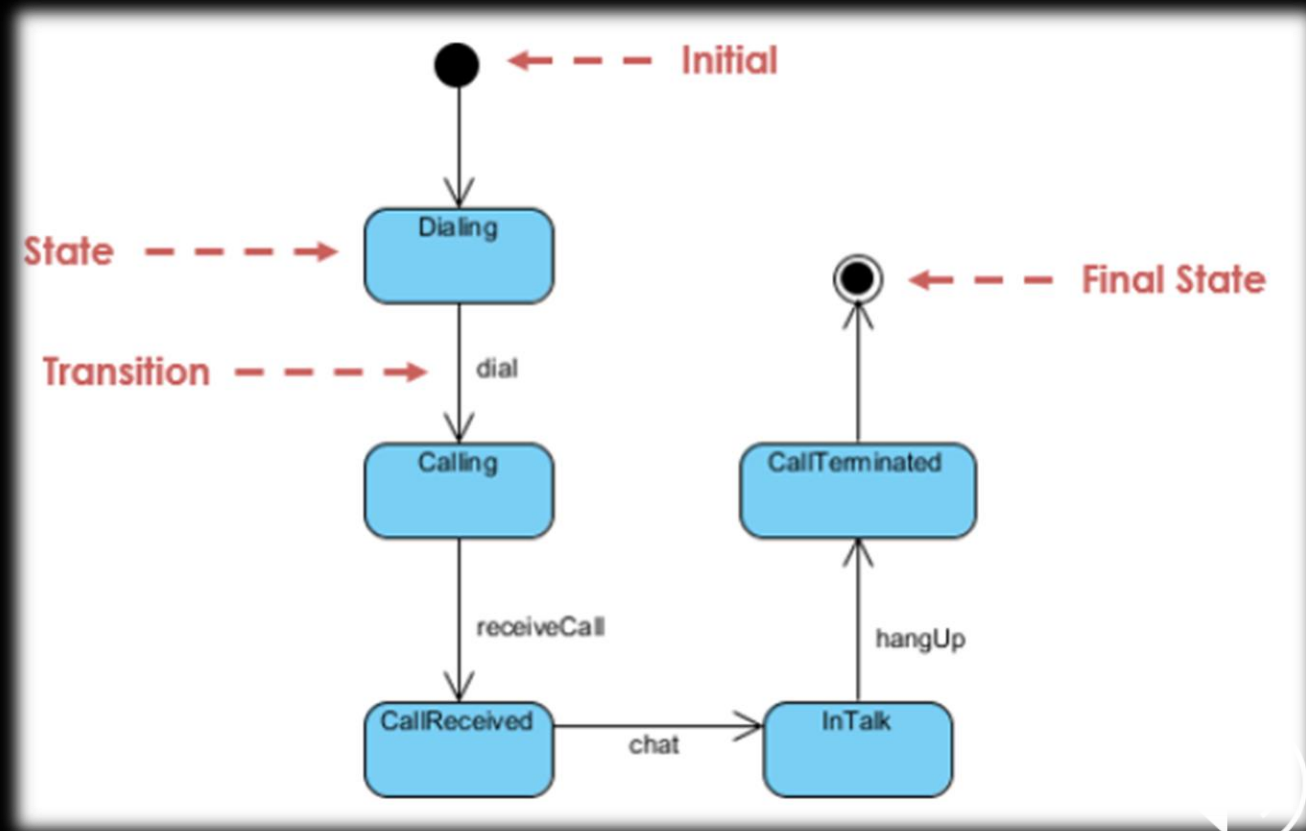
EVENT-DRIVEN MODELING

- »» Real-time systems are often event-driven, with minimal data processing. For example, a landline phone switching system responds to events such as 'receiver off hook' by generating a dial tone.
- »» Event-driven modeling shows how a system responds to external and internal events.
- »» It is based on the assumption that a system has a finite number of states and that events (stimuli) may cause a transition from one state to another.



STATE MACHINE MODELS

- Model the behaviour of the system in response to external and internal events.
- Show the system's responses to stimuli so are often used for modelling real-time systems.
- Show system states as nodes and events as arcs between these nodes. When an event occurs, the system moves from one state to another.
- Statecharts are an integral part of the UML and are used to represent state machine models.





MODEL-DRIVEN ENGINEERING

- »» Model-driven engineering (MDE) is an approach to software development where models rather than programs are the principal outputs of the development process.
- »» The programs that execute on a hardware/software platform are then generated automatically from the models.
- »» Proponents of MDE argue that this raises the level of abstraction in software engineering so that engineers no longer have to be concerned with programming language details or the specifics of execution platforms.





MODEL DRIVEN ARCHITECTURE

- »» Model-driven architecture (MDA) was the precursor of more general model-driven engineering
- »» MDA is a model-focused approach to software design and implementation that uses a subset of UML models to describe a system.
- »» Models at different levels of abstraction are created. From a high-level, platform independent model, it is possible, in principle, to generate a working program without manual intervention.





TYPES OF MODEL

- A computation independent model (CIM)
 - These model the important domain abstractions used in a system. CIMs are sometimes called domain models.
- A platform independent model (PIM)
 - These model the operation of the system without reference to its implementation. The P I M is usually described using UML models that show the static system structure and how it responds to external and internal events.
- Platform specific models (PSM)
 - These are transformations of the platform-independent model with a separate PSM for each application platform. In principle, there may be layers of PSM, with each layer adding some platform-specific detail.





ADOPTION OF MDA

- »» A range of factors has limited the adoption of MDE/MDA Specialized tool support is required to convert models from one level to another
- »» There is limited tool availability and organizations may require tool adaptation and customisation to their environment
- »» For the long-lifetime systems developed using MDA, companies are reluctant to develop their own tools or rely on small companies that may go out of business





AGILE METHODS AND MDA

- »» The developers of MDA claim that it is intended to support an iterative approach to development and so can be used within agile methods.
- »» The notion of extensive up-front modeling contradicts the fundamental ideas in the agile manifesto and agile developers don't feel comfortable with model-driven engineering.
- »» If transformations can be completely automated and a complete program generated from a PIM, then, in principle, MDA could be used in an agile development process as no separate coding would be required.



